Software Engineering: REST APIs and Authentication

A REST API exposes resources through HTTP. A resource is identified by a URI, and representations are exchanged using request and response messages. A client sends a request to a server; the server validates and processes it, then returns a status code and, when appropriate, a representation such as JSON. REST is an architectural style, not a particular programming language or vendor product.

HTTP methods communicate the requested operation. GET retrieves a representation and should not change server state. POST submits data or requests creation of a subordinate resource. PUT replaces a resource representation and is ordinarily idempotent: repeating the same request has the same intended effect. PATCH applies a partial update. DELETE requests removal. Servers should document method behavior and validate request bodies before changing data.

HTTP status codes report the outcome. The 2xx class indicates success: 200 means a request succeeded, 201 means a resource was created, and 204 means success with no response body. The 4xx class indicates a client-side problem: 400 is a malformed or invalid request, 401 means valid authentication credentials are missing or unacceptable, 403 means the caller is authenticated but lacks permission, and 404 means the resource was not found. The 5xx class indicates that the server failed to fulfill an otherwise valid request. A status code alone does not explain every business rule, so APIs commonly return a structured error body too.

Authentication establishes who a caller is; authorization decides what that caller may do. A JWT access token is a signed set of claims that a server can validate. A typical API verifies its signature, accepted algorithm, expiry, and subject before loading the corresponding account. The subject identifies the authenticated principal; a client-supplied user identifier is not proof of identity. Authorization must still check ownership or permissions for each requested resource. Possessing a valid token does not grant access to every document.

A typical API request and response lifecycle begins when a client sends an HTTP method, URI, headers, and optional body. A router matches the URI and method. Authentication middleware or a dependency validates credentials and establishes the principal. Authorization checks that principal against the target resource. Input validation checks types and constraints. The application executes its service and database work, commits successful changes, and serializes a response with an appropriate status code and headers. Errors should be mapped to clear client responses while sensitive internal details stay in server logs.

For a protected document question endpoint, the authenticated account should be derived from the verified token. The service should first load the conversation and verify its owner, then resolve the conversation’s configured document scope. Only after these checks should it query document evidence. This ordering makes identity and authorization independent of the wording of the question and prevents a request from selecting another account’s documents by changing a JSON field.